Learning Experience F.3

Ethan Davidson and Patrick Rooney

ECE 2544

Fall 2024

Design Changes

We did not make any changes to the function unit design from Learning Experience F.2 because our function unit from Learning Experience F.2 already had all of our desired functionality and was completely compatible with our branch on non-negative instruction we added in Learning Experience F.3. In the control unit, we made changes to the next_pc mux to allow Branch On Non-Negative to change the Program Counter. We also added a branch adder circuit, which consists of sixteen full-adders, named branch_adder module, to perform the arithmetic of the branch offset. This design change was necessary because our current design from Learning Experience F.2 could not support the performance of the branch offset needed in the branch on non-negative instruction. These were the only changes made in Learning Experience F.3.

Opcodes

Operation	15	14	13	12	11	10	9	Function Select				MB	MD	RW	MW	PL	JB	BC
Add	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0
Subtract	0	0	0	0	0	0	1	0	0	0	1	0	0	1	0	0	0	1
Negative A	1	0	0	0	0	1	0	0	0	1	0	1	0	1	0	0	0	0
Mult 4	0	0	0	1	0	1	1	1	0	1	1	0	0	1	0	0	0	1
Mov A	0	0	0	0	1	0	1	0	1	0	1	0	0	1	0	0	0	1
Rem 8	0	0	0	1	1	0	0	1	1	0	0	0	0	1	0	0	0	0
Nor	0	0	0	0	1	1	1	0	1	1	1	0	0	1	0	0	0	1
Negative B	0	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	1
Dec A	1	0	0	0	1	0	0	0	1	0	0	1	0	1	0	0	0	0
Not A	1	0	0	0	1	1	0	0	1	1	0	1	0	1	0	0	0	0
And	0	0	0	1	0	0	0	1	0	0	0	0	0	1	0	0	0	0
Nand	0	0	0	1	0	0	1	1	0	0	1	0	0	1	0	0	0	1
Not B	0	0	0	1	0	1	0	1	0	1	0	0	0	1	0	0	0	0
ADD I	1	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0
SUB I	1	0	0	0	0	0	1	0	0	0	1	1	0	1	0	0	0	0
NOR I	1	0	0	0	1	1	1	0	1	1	1	1	0	1	0	0	0	1
Negative B	0	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	1
Decrement A	1	0	0	0	1	0	0	0	1	0	0	1	0	1	0	0	0	1
Not B	0	0	0	0	0	1	1	0	0	1	1	0	0	1	0	0	0	1
BRNN	0	1	1	0	1	0	1	0	1	0	1	0	1	1	1	0	1	1

Figure 1.1 Opcode Table

Simulation and Discussion

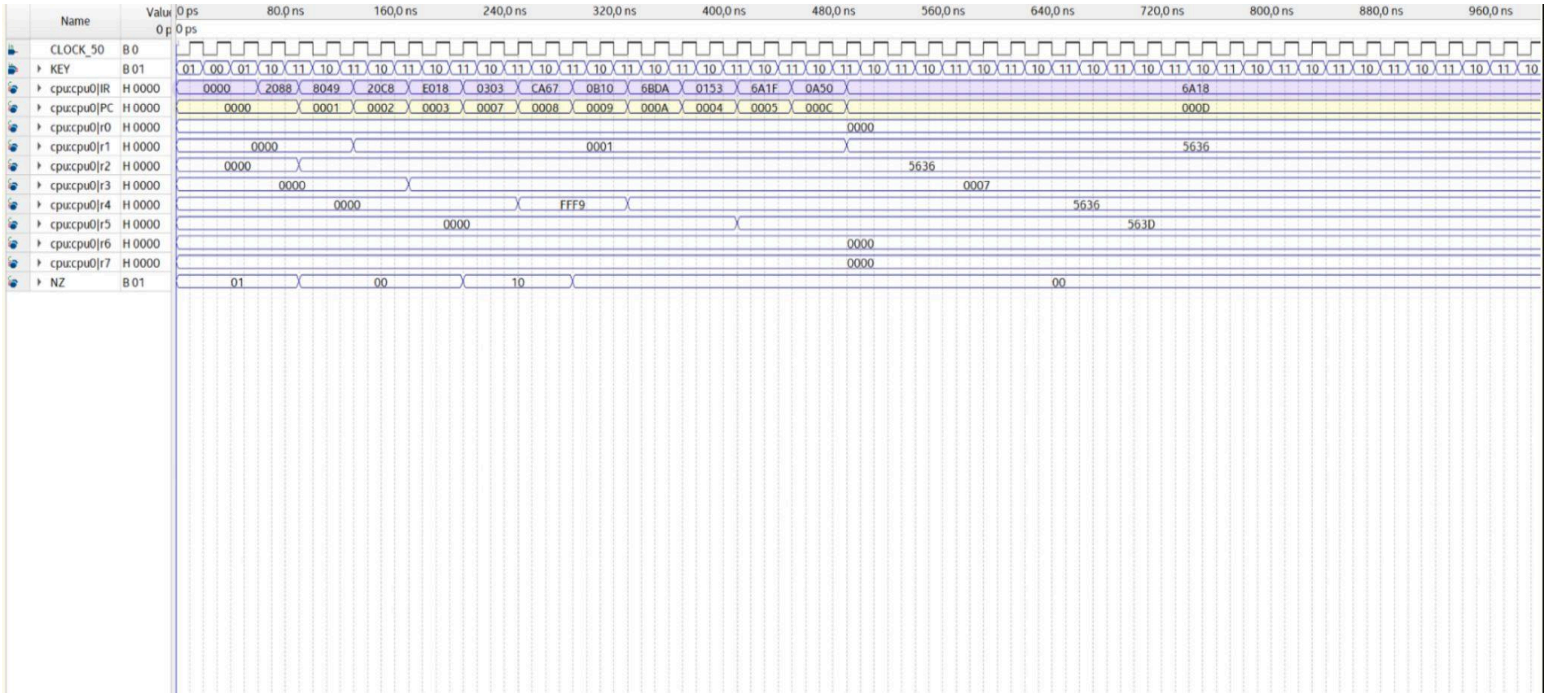

Figure 1.2 Annotated Simulation

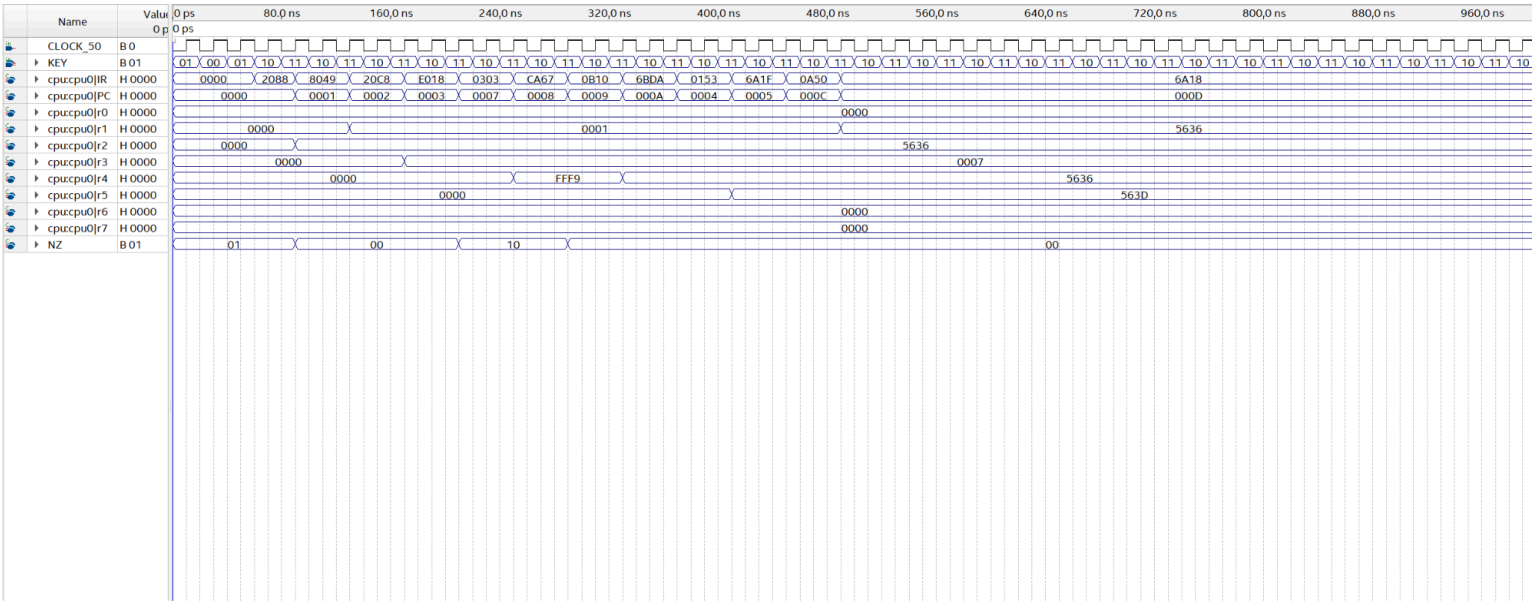

Figure 1.3 Simulation

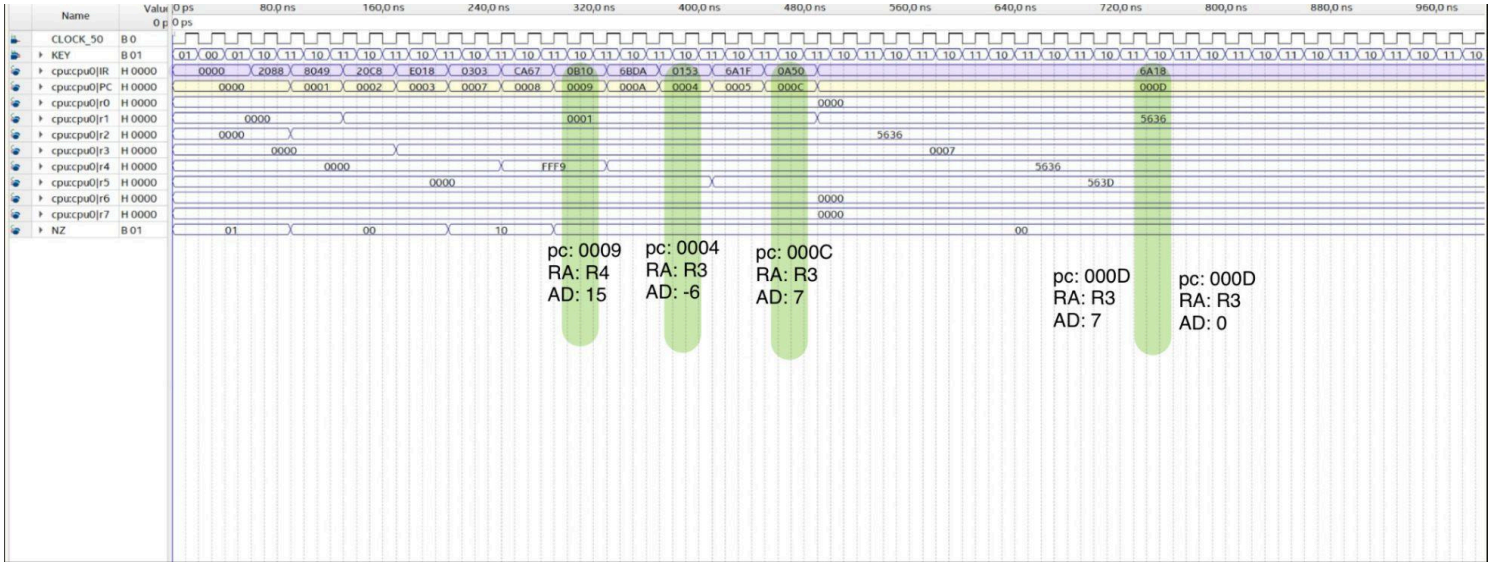

Figure 1.4 annotated BRNN simulation

In our annotated simulation, shown in *Figure 1.2*, the purple highlighter is instruction memory and the yellow highlighter is the program counter. When RA contains a value less than zero nothing happens because it only branches on non-negative values of RA. When RA ≥ 0 and AD>0 it branches forward by the magnitude of AD. When RA ≥ 0 and AD<0 it branches backward by the magnitude of AD. When RA ≥ 0 and AD=0 it starts an infinite loop effectively terminating the program. All other functions of the program counter still work properly. In our figure 1.4, we annotated our simulation to include specific times when BRNN happens, the program control, the source register A, and the AD value of the branch offset. The last green highlighted area is when BRNN occurs twice at the end of the simulation which is why there are two annotations.